

Data Cleaning & EDA Reference

Prepared by **Kawchar Husain**

[Kaggle](#) | [LinkedIn](#)

Loading & First Look

```
import pandas as pd
import numpy as np

df = pd.read_csv('filename.csv')

# First look
df.head()
df.tail()
df.shape
df.info()
df.describe()
df.columns
df.dtypes
```

Data Cleaning

Missing Values

```
# Detect missing values
df.isnull().sum()
df.isnull().sum() / len(df) * 100

# Visualize
import seaborn as sns
sns.heatmap(df.isnull(), cbar=False)

# Drop rows with missing values
df.dropna()
df.dropna(subset=['column'])

# Fill missing values
df['col'].fillna(value)
df['col'].fillna(df['col'].mean())
df['col'].fillna(df['col'].median())
df['col'].fillna(df['col'].mode()[0])
df['col'].fillna(method='ffill')
df['col'].fillna(method='bfill')
```

Duplicates

```
# Detect and remove
df.duplicated()
df.duplicated().sum()
df.drop_duplicates()
df.drop_duplicates(subset=['col'])
```

Text Cleaning

```
# Case conversion
df['col'].str.lower()
df['col'].str.upper()

# Remove whitespace
df['col'].str.strip()
```

```
df['col'] = df['col'].str.strip()

# Split text
df['col'].str.split(',')
df['col'].str.split(',').str[0]
```

Type Conversion

```
df['col'].astype('int')
df['col'].astype('float')
df['col'].astype('str')
pd.to_numeric(df['col'])
pd.to_datetime(df['col'])
```

Selection & Filtering

Selecting Data

```
# Select columns
df['column']
df[['col1', 'col2']]

# Select rows
df.iloc[0]
df.iloc[0:5]
df.loc[0]

# Specific rows & columns
df.loc[0:5, ['col1', 'col2']]
df.iloc[0:5, 0:2]
```

Filtering Rows

```
# Single condition
df[df['column'] > value]
df[df['column'] == value]

# Multiple conditions
df[(df['col1'] > val) & (df['col2'] < val)]
df[(df['col1'] == val) | (df['col2'] == val)]

# Filter by list
df[df['column'].isin([val1, val2])]

# Filter by pattern
df[df['column'].str.contains('pattern')]

# Non-null values
df[df['column'].notna()]
```

EDA with Pandas

Basic Statistics

```
df['col'].mean()
df['col'].median()
df['col'].mode()[0]
df['col'].std()
df['col'].min()
df['col'].max()
df['col'].sum()
df['col'].count()
```

Value Counts

```
df['col'].value_counts()
df['col'].nunique()
df['col'].unique()
```

Group By & Aggregation

```
# Single aggregation
df.groupby('col').mean()
df.groupby('col').sum()
df.groupby('col').count()
df.groupby('col').size()

# Multiple aggregations
df.groupby('col').agg(['mean', 'sum', 'min'])

# Group by multiple columns
df.groupby(['col1', 'col2']).mean()

# Specific column aggregation
df.groupby('col')['value_col'].mean()
```

Correlation & Sorting

```
# Correlation
df.corr()
df['col1'].corr(df['col2'])

# Sorting
df.sort_values('col')
df.sort_values('col', ascending=False)
df.sort_values(['col1', 'col2'])
```

Outlier Detection

IQR Method

```
# Calculate quartiles
Q1 = df['col'].quantile(0.25)
Q3 = df['col'].quantile(0.75)
IQR = Q3 - Q1

# Define outlier bounds
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

# Find outliers
outliers = df[(df['col'] < lower) |
              (df['col'] > upper)]

# Remove outliers
df_clean = df[(df['col'] >= lower) &
              (df['col'] <= upper)]
```

Visualization

Pandas Built-in

```
# Histogram
df['col'].hist()
df['col'].hist(bins=20)

# Box plot
df.boxplot(column='col')
```

```
df.boxplot(column='val', by='category')

# Bar plot
df['col'].value_counts().plot.bar()

# Scatter & Line
df.plot.scatter(x='col1', y='col2')
df.plot.line()
```

Matplotlib

```
import matplotlib.pyplot as plt

# Histogram
plt.hist(df['col'], bins=20)
plt.xlabel('Column')
plt.ylabel('Frequency')
plt.title('Distribution')
plt.show()

# Scatter plot
plt.scatter(df['col1'], df['col2'])
plt.xlabel('Column 1')
plt.ylabel('Column 2')
plt.show()
```

Seaborn

```
# Distribution
sns.histplot(df['col'], bins=20)
sns.histplot(data=df, x='col', hue='category')

# Box plot
sns.boxplot(data=df, x='category', y='value')

# Count plot
sns.countplot(data=df, x='col')

# Heatmap (correlation)
sns.heatmap(df.corr(), annot=True,
            cmap='coolwarm')

# Pair plot
sns.pairplot(df)
```

Saving Data

```
df.to_csv('filename.csv', index=False)
df.to_excel('filename.xlsx', index=False)
```

Tips & Best Practices

- Always check for missing values before analysis
- Understand your data types (`df.dtypes`)
- Use `.describe()` for quick statistical overview
- Visualize data before and after cleaning
- Keep a copy of original data before modifications
- Check for duplicates in your dataset
- Handle outliers based on domain knowledge

For more information: pandas.pydata.org/docs